

Introduction to Retrieval-Augmented Generation (RAG)

Sample Knowledge Base Document

Retrieval-Augmented Generation, commonly called RAG, is an architecture that combines information retrieval with a language model. Instead of relying only on information stored in the model's parameters, a RAG system retrieves relevant external documents and provides their content to the language model as context.

1. What is RAG?

A typical RAG pipeline has three major stages: ingestion, retrieval, and generation. During ingestion, documents are loaded, cleaned, split into smaller chunks, converted into vector embeddings, and stored in a vector database. During retrieval, a user's question is converted into an embedding and compared with stored document embeddings. The most relevant chunks are then returned. During generation, the retrieved context and the user's question are sent to a language model to produce an answer.

2. Main Components

Component	Purpose
Document Loader	Reads PDFs, text files, web pages, or other knowledge sources.
Text Splitter	Breaks large documents into smaller chunks suitable for retrieval.
Embedding Model	Converts text into numerical vectors representing semantic meaning.
Vector Database	Stores embeddings and enables similarity searches.
Retriever	Finds the document chunks most relevant to a user question.
Language Model	Uses the retrieved context to generate the final response.

3. Example RAG Workflow

Suppose a company has a 100-page employee handbook. A user asks: **“How many days of annual leave can an employee take?”** The system first searches the vector database for chunks related to annual leave. It may retrieve sections describing leave entitlement and approval rules. Those chunks are placed into the model's context, allowing the model to answer using the company's handbook rather than guessing.

4. Chunking

Chunking is the process of dividing a document into smaller pieces before creating embeddings. A common strategy is to use chunks of roughly 300–800 tokens with some overlap between adjacent chunks. The ideal size depends on the document structure and the type of questions users will ask.

5. Embeddings and Vector Search

An embedding represents text as a numerical vector. Texts with similar meanings tend to have vectors that are close together in the embedding space. Vector databases such as Chroma, Pinecone, FAISS, and Weaviate can be used to store and search these vectors.

6. Example Technology Stack

Backend: FastAPI

Framework: LangChain

Embeddings: Hugging Face or another embedding provider

Vector Database: Chroma

LLM: OpenAI or another compatible language model

Source: PDF documents

7. Important RAG Concepts

Key concepts to understand include document ingestion, metadata, chunk size, chunk overlap, embeddings, similarity search, top-k retrieval, reranking, context windows, prompt construction, hallucination reduction, and evaluation. A good RAG system should retrieve relevant evidence and avoid adding unsupported information.

8. Practice Questions

1. Why do we split a PDF into chunks?
2. What is an embedding?
3. Why is a vector database useful in RAG?
4. What happens when a user submits a question?
5. What is the difference between retrieval and generation?
6. How can poor chunking affect retrieval quality?

End of Sample Knowledge Base